Data Generation Pipeline

1. Dataset Foundation

MS-COCO 2017 validation split (4,000 images) is used as the clean base. It was chosen specifically for its pixel-level instance segmentation masks, which enable precise object extraction for copy-move and splicing without approximation. Images span 80 semantic categories across diverse real-world scenes, ensuring varied camera noise profiles and lighting conditions.

2. Tampering Strategy

- The tampering pipeline combines **spatial manipulations** (e.g., object insertion or duplication) with **forensic artifact manipulations** (e.g., noise and compression inconsistencies) to simulate realistic image editing scenarios.
- This design ensures that each manipulation introduces **detectable forensic signals**, encouraging the model to learn both **semantic inconsistencies and low-level statistical artifacts** rather than relying only on visible patterns.

3. Object Selection Algorithm

Before applying any tampering, the pipeline first identifies a suitable object or region to manipulate. Random selection often produces unrealistic edits (edge clipping, weak forensic signals, or implausible object placement). To avoid this, objects are selected using a scoring-based filtering strategy.

For object-based manipulations (Copy-Move, Inpainting, Splicing), each COCO object instance is evaluated using four criteria:

- **Semantic Tier** – Object categories are ranked by suitability for manipulation. Solid objects such as *cars, animals, and people* are preferred, while context-dependent objects (e.g., traffic lights) are excluded.

- **Area Ratio** – Optimal object size is 2–12% of the image area, ensuring the manipulation is neither visually trivial nor unrealistically dominant.

- **Mask Solidity** – Objects with dense masks (solidity > 0.55) are preferred to avoid sparse structures that produce poor composites.

- **Boundary Safety** – Objects near image borders (<1% distance from edge) are excluded to prevent truncated masks.

All candidates are scored and ranked, and one object is randomly sampled from the top three. This maintains manipulation quality while introducing diversity across the dataset.

For region-based manipulations (Frequency-domain editing and Local Manipulation), object masks are not required. Instead, regions are selected using Laplacian variance, which identifies texture-rich areas that produce stronger forensic artifacts after manipulation.

Using COCO segmentation masks enables precise pixel-level extraction instead of rectangular crops, producing cleaner composites and more realistic forgeries.

4. Tampering Techniques Used

4.1 Copy-Move

What it does?

- Copies an object from one location within the same image and pastes it at a different location in the same image

Object Selection

- Uses the scoring algorithm (semantic tier, area ratio, solidity, boundary safety) to pick the best candidate object
- COCO segmentation mask is extracted and resized to pixel coordinates

Target Location Search

- 100 random candidate positions are generated across the image
- Each candidate is filtered through 3 constraints:
    - **Histogram similarity** - 2D HSV histogram (50×60 bins) of source vs candidate region must score >0.35 - ensures matching lighting and colour temperature
    - **Vertical displacement** - target must be within −5% to +20% of source Y-position; mimics real-world perspective depth
    - **IoU check** - target must not overlap any existing COCO object (IoU < 5%) - prevents implausible placements
- Best scoring candidate is selected as the paste location

Geometric Transform

- **Random rotation ±15**° applied to the extracted patch
- Scale factor computed based on vertical displacement - object placed lower in frame is scaled up (max 1.5×), higher is scaled down (min 0.3×)
- This approximates perspective scaling in real scenes.

Blending

- COCO mask is eroded (2 iterations) to pull edges inward - avoids halo effect at boundaries
- **Gaussian blur (7×7)** applied to eroded mask - creates smooth feathered alpha channel
- Synthetic shadow generated (31×31 blur, 20% opacity) and subtracted from background before compositing
- Used **Alpha Blending** - Final blend - `alpha × src_patch + (1 - alpha) × shadowed_background`

Ground Truth Mask

- Both source and destination regions are marked as tampered (255)
- Dual-region mask means the model can learn from either the copied-from or pasted-to location

How the Model Learns From This Manipulation

EfficientNet (Semantic Branch) - Learns **structural repetition patterns** and unusual spatial relationships created by duplicated objects.

SRM Noise Branch - Detects **noise inconsistencies and resampling artifacts** introduced during geometric transformations and blending.

ELA Branch - Highlights **compression inconsistencies** near manipulated boundaries where recompression affects pixel statistics.

4.2 AI Inpainting (LaMa)

What it does?

- Removes a selected object from the image and fills the gap with AI-generated background content - simulating the "object erasure" forgery common in journalism and evidence manipulation

Why LaMa?

- **LaMa (Large Mask Inpainting)** uses Fast Fourier Convolutions (FFC) to model long-range spatial dependencies - unlike standard CNN inpainting, it fills large missing regions with globally consistent texture and structure, making its output far harder to detect than older methods

Object Selection

- Same scoring algorithm (semantic tier, area ratio, solidity, boundary safety)
- COCO segmentation mask extracted and resized to pixel coordinates

Mask Preparation

- The binary mask is dilated by **15 pixels** before inpainting to ensure that boundary pixels belonging to the removed object are fully covered. Without dilation, residual edge pixels can remain visible after inpainting, creating unrealistic artifacts.
- Dilated mask converted to PIL format for LaMa compatibility

Inpainting

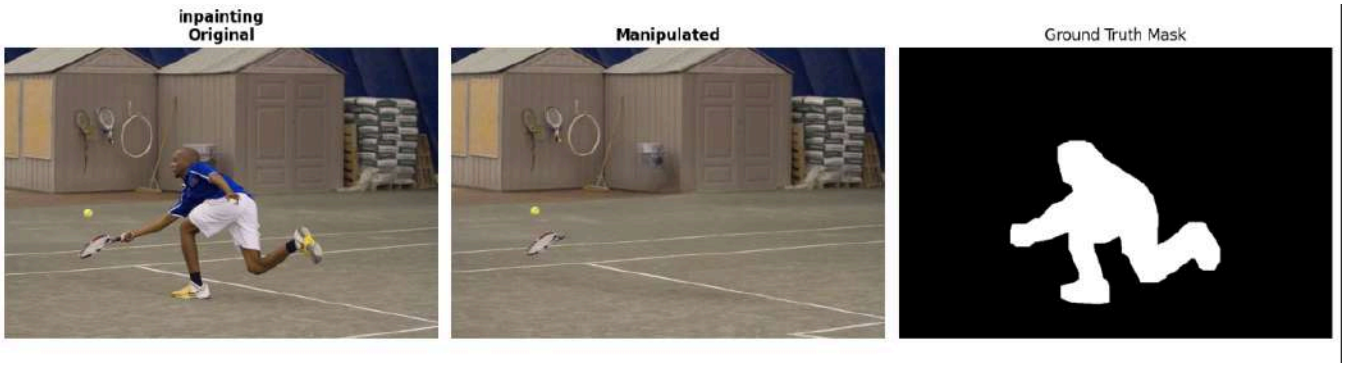
- LaMa fills the masked region with photorealistic background content
- Quality check: If the mean pixel difference between the inpainted image and the original image is below a threshold of 1.0, the sample is discarded. This filters out cases where the inpainting model fails to modify the masked region.

Ground Truth Mask

- The dilated object region is marked as tampered (255)
- Mask represents where the original content was erased and synthetically replaced

How the Model Learns From This Manipulation

- **EfficientNet (Semantic Branch)** - Learns to spot regions that are semantically "too perfect" - missing the natural irregularities of a real scene
- **SRM Noise Branch** - Detects inconsistencies in noise residual statistics within the inpainted region, since synthetically generated pixels do not follow the natural sensor noise patterns present in the original image
- **ELA Branch** - Inpainted regions respond differently to compression re-encoding, producing a distinct ELA signature compared to authentic surrounding pixels



4.3 Frequency Domain (DCT)

What it does?

- Modifies **JPEG compression coefficients** in a selected region to simulate professional codec-level editing. Although the image appears visually unchanged, this process introduces subtle frequency-domain artifacts and compression inconsistencies (e.g., double-compression fingerprints) that can be detected statistically by forensic analysis methods.

Region Selection

- No object mask needed - region selected by **Laplacian variance**
- 50 random candidate regions sampled, highest variance region selected - texture-rich areas carry the most forensic information after manipulation

JPEG Quality Estimation

- Original JPEG quality factor estimated by reading the image **quantization table**
- Re-encoding uses the same quality level - keeps the double-compression artifact subtle and realistic

DCT Block Processing

- The selected region is divided into **8×8 blocks**, following the standard JPEG compression structure.
- Mid-frequency coefficients are slightly increased to simulate artificial sharpening, while high-frequency coefficients are reduced to mimic noise suppression.
- These modifications alter the statistical distribution of **DCT coefficients** without creating visible artifacts in the image.

Double Compression

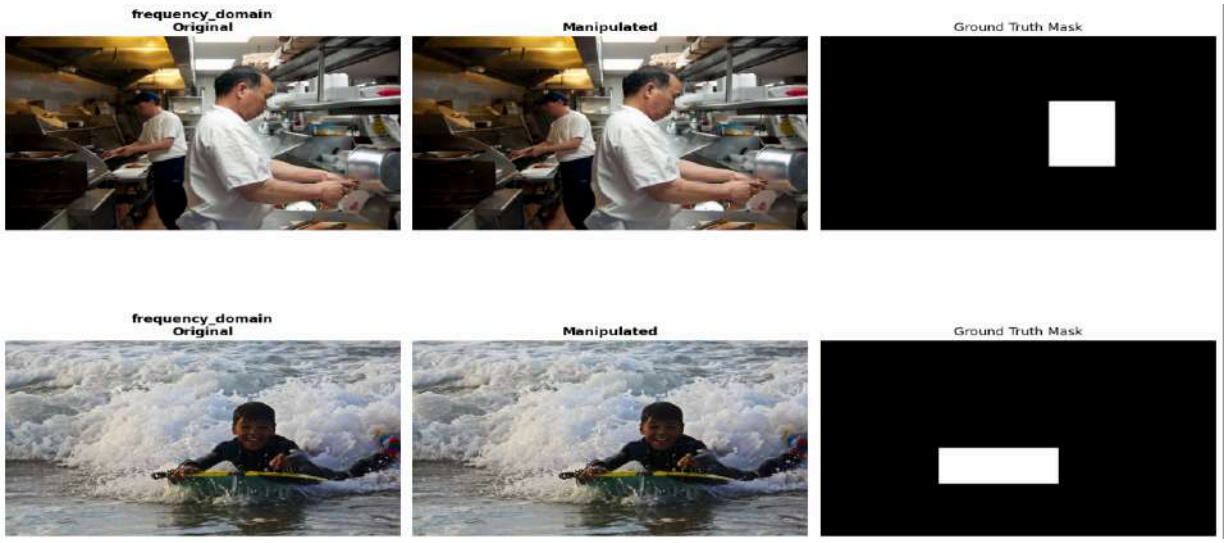
- Region is JPEG-encoded and decoded twice after DCT modification
- The manipulated region undergoes two JPEG compression cycles, while the rest of the image only experiences one. This produces a characteristic **double-compression signature** in the DCT coefficient distribution.

Ground Truth Mask

- The selected rectangular region is marked as tampered (255)
- Soft mask also generated via Gaussian blur for boundary-aware training

How the Model Learns From This Manipulation

- **EfficientNet (Semantic Branch)** - Learns subtle visual texture differences between the manipulated region and surrounding authentic content
- **SRM Noise Branch** - Detects the altered noise distribution caused by coefficient modification and double compression
- **ELA Branch** - **Primary detector** here - double-compressed regions produce a distinctly different ELA response, making this the strongest branch for frequency domain forgeries



4.4 Local Colour Manipulation

What it does?

- Selectively edits colour properties (**brightness, contrast, saturation** etc.) in a local region of the image - simulating the most common real-world photo editing done in tools like Photoshop or Lightroom

Region Selection

- A random rectangular region (**15–40%** of image dimensions) is selected

- No object mask needed - works on any area of the image

6 Manipulation Types (one picked at random)

- **Brightness** - shifts pixel brightness up or down (±15–40) in the V channel
- **Contrast** - stretches or compresses the total range around the mean value
- **Saturation** - makes colours more vivid or washed out by scaling the S channel
- **Sharpening** - applies an unsharp mask kernel to enhance edges and fine detail
- **Hue Shift** - rotates the colour of the region (e.g. red object appears blue)
- **Combined** - applies brightness + saturation + slight hue shift together - closest to real professional retouching

Blending

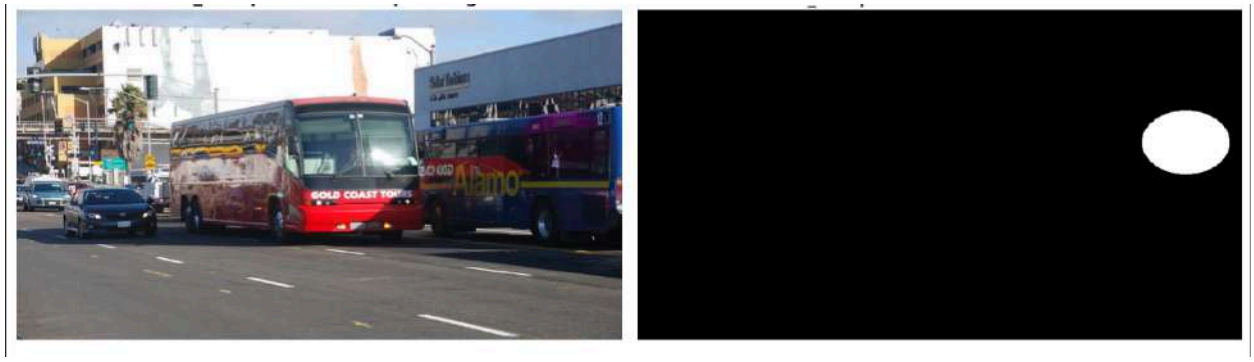
- A Gaussian-feathered ellipse mask is used to blend the edit back into the image
- Mimics the soft brush falloff of real editing tools - no hard rectangular boundary visible

Ground Truth Mask

- Ellipse shape is used for the mask - not the bounding rectangle
- Ensures the model is trained on exactly the pixels that were changed, not the surrounding unedited area

How the Model Learns From This Manipulation

- **EfficientNet (Semantic Branch)** - Detects visible colour inconsistencies between the edited region and surrounding authentic content
- **SRM Noise Branch** - Colour operations **alter the noise distribution** inside the region - SRM picks up the statistical boundary where noise pattern changes
- **ELA Branch** - **Re-encoding affects the edited region differently**, producing a visible ELA response at the manipulation boundary



4.5 Cross-Domain Splicing

What it does?

- Takes a real COCO object and pastes it onto a completely different background image from the **DTD (Describable Textures Dataset)** - creating a fundamental camera-source mismatch where the object and background were captured by different cameras in different lighting environments

Why DTD as Background?

- DTD contains 5,640 texture images - grass, sky, water, wood, fabric, sand etc.
- Semantically distinct from COCO scenes - maximises the domain gap between object and background
- Stronger forensic signal because the two regions have completely different noise profiles and colour science

Object Selection

- Same scoring algorithm (semantic tier, area ratio, solidity, boundary safety)
- COCO segmentation mask extracted for precise object boundary

Placement

- Object rescaled to fit within **1/3** of background dimensions - prevents large objects like buses or elephants from covering the entire background
- Random position selected on the background image

Blending

- **Poisson blending** (cv2.seamlessClone) is used to integrate the object with the background.
- This method adjusts color gradients at the boundary, preventing visible seams that occur with simple alpha blending.
- If Poisson blending fails, a Gaussian-feathered alpha blend is used as a fallback.

Ground Truth Mask

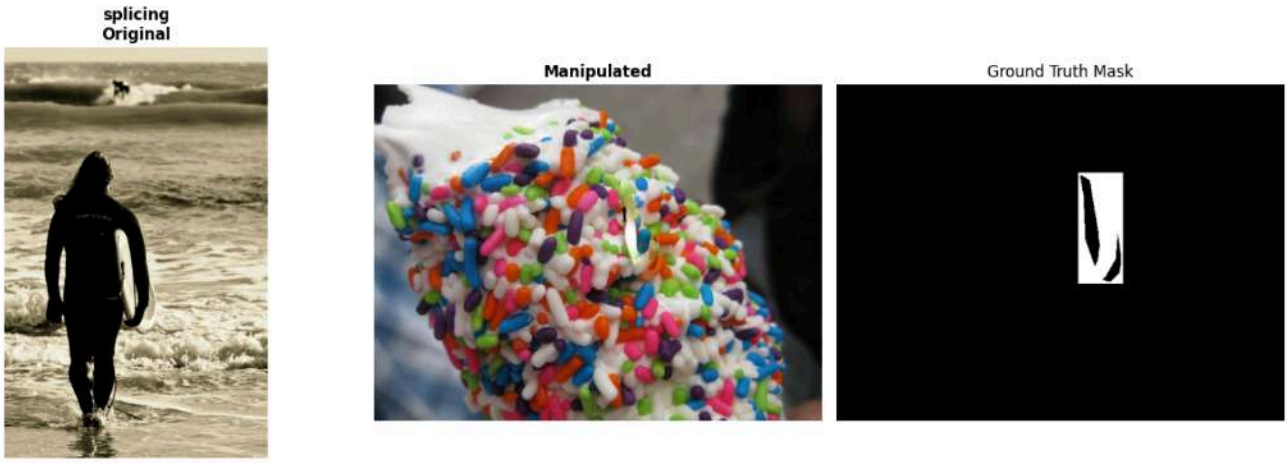
- The pasted object region on the background is marked as tampered (255)
- Mask follows the exact COCO segmentation shape - not a bounding box

Authentic Background Tracking

- Each background image used in a splice is also recorded as label=0 (authentic)
- This ensures the model encounters the same background in both clean and tampered contexts, preventing it from simply memorizing background textures.

How the Model Learns From This Manipulation

- **EfficientNet (Semantic Branch)** - Detects semantic inconsistency between the pasted object and the background scene context
- **SRM Noise Branch** - **Detects inconsistencies in noise residual statistics** between the object and background. Since the two regions originate from different images, their noise patterns and high-frequency statistics differ, creating detectable boundaries.
- **ELA Branch** - The object and background may have different JPEG compression histories, producing distinct ELA responses at the splice boundary.



5. Dataset Structure

- **Metadata Organization** – Each manipulation technique has a dedicated CSV file containing image path, label (authentic/tampered), manipulation type, and ground-truth mask path. This enables efficient data loading and per-technique evaluation.
- **Dataset Composition** – The dataset includes **4,000 authentic COCO images**, **1,000 authentic background images**, and **1,000 tampered samples per manipulation technique**, ensuring balanced representation across manipulation types.
- **Ground Truth Masks** – Every tampered image is paired with a **pixel-level manipulation mask**, enabling both classification training and interpretability analysis (e.g., GradCAM comparison).

Model Training Pipeline - ForgeryDetector

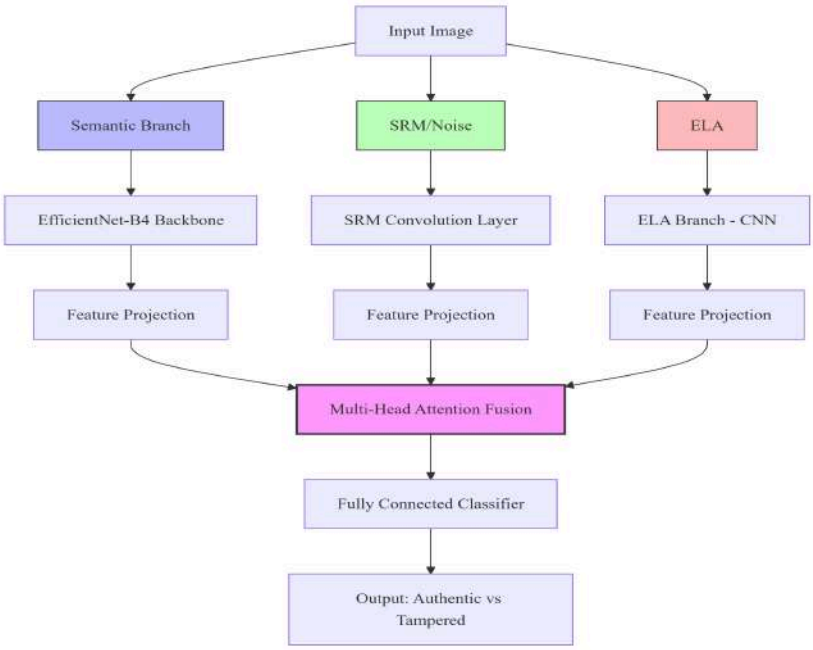
1. Overview

ForgeryDetector is a professional-grade image forensics framework designed to identify various types of digital manipulations, including splicing, copy-move, inpainting, and frequency-domain modifications. The system leverages a multi-stream deep learning architecture that captures complementary forensic signals across semantic, noise-residual, and compression-artifact domains. By integrating these signals through an attention-based fusion mechanism, the model achieves high-precision detection even for subtle, localized tampering.

2. Technical Approach

The core design philosophy rests on the observation that different tampering techniques leave distinct traces in different image layers. While semantic inconsistencies are often visible to the naked eye, noise-level disturbances and JPEG compression shifts require specialized extraction filters. ForgeryDetector addresses this by processing images through three specialized branches,ensuring that no forensic signal is overlooked during the classification process.

3. Model Architecture



3.1 Architecture Overview

The **ForgeryDetector** uses a three-branch multi-stream architecture to capture different forensic signals present in tampered images. Relying only on semantic CNN features is often insufficient because many manipulations preserve the overall scene structure while altering subtle statistical properties such as noise patterns or compression artifacts.

To address this, the model analyzes each image from three complementary perspectives: **semantic structure, noise residuals, and compression artifacts**.

- **Branch 1 (Semantic)** - Extracts high-level visual semantic features from a pretrained deepCNN.
- **Branch 2 (Noise)** - Captures pixel-level noise residuals via handcrafted steganalysis filters.
- **Branch 3 (ELA)** - Identifies JPEG compression artifacts via Error Level Analysis maps.

3.2 Semantic Branch — EfficientNet-B4

- **Purpose** – The semantic branch acts as the **visual perception component** of the model. It processes the raw RGB image to detect **structural and contextual inconsistencies** such as duplicated objects, mismatched edges, unnatural textures, inconsistent lighting, or objects that do not logically belong in the scene. Unlike forensic branches that focus on compression artefacts or noise residuals, this branch evaluates the **semantic coherence and structural integrity** of the image.
- **Backbone -** EfficientNet-B4 pretrained on ImageNet-1K.
- **Feature Extraction -** All layers except the classification head are utilized, producing a 1792-dimensional feature vector.
- **Fine-tuning Strategy -** To prevent catastrophic forgetting and preserve the rich semantic features learned from ImageNet, only the last two MBConv blocks (indices 7 and 8) are unfrozen. The rest of the backbone remains fixed during the initial training phases.
- **Output -** A 1792-dimensional global average pooled representation.
- **Rationale -** EfficientNet-B4 provides a superior balance between computational efficiency and representational power. Its compound scaling is particularly effective at capturing themulti-scale structural inconsistencies often introduced by splicing and large-scale copy-move operations.

3.3 Noise Branch — SRM + CNN

- **SRM (Steganalysis Rich Model) -** The branch begins with three fixed, non-trainable 5×5convolutional filters derived from steganalysis literature:

  F1 - Second-order central difference (Laplacian-like), scaled by 1/4.

  F2 - High-order difference filter, scaled by 1/12.

  F3 - Simple horizontal second difference, scaled by 1/2.

- **SRM Implementation** - All SRM filters are initialized with `requires_grad = False`, meaning their weights are **not updated during backpropagation**. They remain fixed throughout training and function purely as **deterministic residual extractors** rather than learnable parameters. This is important because SRM filters are carefully designed to reveal **pixel-level noise patterns and manipulation traces**, and allowing them to update could cause the network to lose this specialized forensic behaviour.
- **Activation -** A **tanh activation** is applied after SRM filtering to normalize residual magnitudes while preserving the **sign of the residuals**.
- **Post-SRM CNN -**

  The residual maps are processed by a lightweight CNN:

  Conv2d(3→32) →BN → ReLU → MaxPool → Conv2d(32→64) → BN → ReLU → MaxPool → Conv2d(64→128) → BN → ReLU → AdaptiveAvgPool(1,1)

  These layers progressively learn patterns in the noise residuals while reducing spatial size, producing a compact representation.

- **Output** - A 128-dimensional noise feature vector.
- **Rationale** - SRM filters are highly sensitive to pixel-level manipulation traces. Local operations often disturb the underlying camera sensor noise patterns, which are invisible to the human eye but detectable through these residual filters.

```
f1 = np.array([[ 0, 0, 0, 0, 0],[ 0,-1, 2,-1, 0],[ 0, 2,-4, 2, 0],[ 0,-1, 2,-1, 0],[ 0, 0, 0, 0, 0]], dtype=np.float32) / 4
f2 = np.array([[-1, 2,-2, 2,-1],[ 2,-6, 8,-6, 2],[-2, 8,-12,8,-2],[ 2,-6, 8,-6, 2],[-1, 2,-2, 2,-1]], dtype=np.float32) / 12
f3 = np.array([[ 0, 0, 0, 0, 0],[ 0, 0, 0, 0, 0],[ 0, 1,-2, 1, 0],[ 0, 0, 0, 0, 0],[ 0, 0, 0, 0, 0]], dtype=np.float32) / 2
```

3.4 ELA Branch — Error Level Analysis CNN

- **Error Level Analysis (ELA)** – ELA is computed by **re-saving the image at JPEG quality Q=75** and calculating the **absolute pixel difference between the original and recompressed image**. The resulting difference map is normalized to highlight compression inconsistencies.
- **What it Captures** – Edited regions often have **different JPEG compression histories** than the surrounding image. This causes **variation in compression error levels**, which appear as intensity differences in the ELA map.

- **ELA CNN** – The ELA map is processed using the same lightweight CNN used in the noise branch:

  `Conv2d(3→32) → BN → ReLU → MaxPool → Conv2d(32→64) → BN → ReLU → MaxPool → Conv2d(64→128) → BN → ReLU → AdaptiveAvgPool`.

- **Output** – A **128-dimensional compression artifact feature vector**.
- **Rationale** – Manipulations such as **splicing or frequency-domain editing** disturb the **JPEG DCT compression statistics**. ELA exposes these inconsistencies, making it effective for detecting composites created from images with **different compression histories**.

3.5 Projection and Attention Fusion

- **Common Embedding Space** – The three branches produce feature vectors of different sizes (**1792, 128, 128**). To combine them, each output is projected into a **shared 512-dimensional space** using linear layers:

  ```
  proj_sem : Linear(1792 → 512)
  proj_noise : Linear(128 → 512)
  proj_ela : Linear(128 → 512)
  ```

  This ensures all branches contribute features with the **same dimensional representation**, making fusion consistent and comparable.

- **Attention Fusion (MHSA)** – Instead of simply concatenating the features, the model uses an **8-head Multi-Head Self-Attention (MHSA)** module on the sequence **[semantic, noise, ela]**. Each branch acts like a **token**, and the attention mechanism learns how much each branch should influence the final representation for a given image.
- **Why Not Concatenation?** – Simple concatenation treats all branches **equally and statically**. However, different manipulations leave different traces. Attention allows the model to **dynamically weight the importance of each branch** depending on the input image.
- **Stabilization** – A **residual connection** and **LayerNorm** are applied after attention to maintain stable gradient flow and normalized feature distributions.
- **Output** – The three attention-refined tokens are **mean pooled** to produce a single **512-dimensional fused representation**.
- **Rationale** – Since manipulation traces vary by technique, **attention fusion enables adaptive feature weighting**, allowing the model to focus on the most informative forensic signal for each case rather than relying on fixed feature concatenation.

3.6 Classifier Head

- **Architecture -** A three-layer MLP:

  Linear(512→256) → ReLU → Dropout(0.3) →Linear(256→64) → ReLU → Dropout(0.15) → Linear(64→2)

- **Output -** 2-class logits representing “Authentic” and “Tampered” states.

3.7 Architecture Summary Table

Component	Type	Input Dim	Output Dim	Trainable
Semantic Branch	EfficientNet-B4 (Partial)	3×224×224	1792	Blocks 7, 8
Noise Branch	SRM + 3-layer CNN	3×224×224	128	Yes
ELA Branch	3-layer CNN	3×224×224	128	Yes

Projection Layers	Linear ×3	1792 / 128 / 128	512	Yes
Fusion	8-head MHA	3×512	512	Yes
Classifier	3-layer MLP	512	2	Yes
Total Params	—	—	20 M	—
Trainable Params	—	—	6.7 M	—

4. Data Preprocessing & Augmentation

The final training pipeline incorporates forensic-aware augmentations to improve generalization.

1. **Resize** - Standardizes input size so it matches the expected input resolution of the **EfficientNet-B4** backbone.

2. **Horizontal/Vertical Flip** - p=0.5 and p=0.2 respectively. Introduces spatial variation to improve **model generalization** and reduce overfitting.

3. **JPEGCompression** - Quality range (60, 95) with p=0.3 Simulates **real-world social media recompression**, helping the model learn compression-related forensic artifacts.

4. **ToTensor** - Conversion to floating point representation.

5. **HighPassFilter** - A Laplacian kernel applied post-tensor with p=0.3. Emphasizes **edge and noise residuals** that often contain manipulation traces.

6. **Normalize** - ImageNet mean and standard deviation. Aligns input statistics with **ImageNet pretraining**, ensuring stable and efficient feature extraction.

ELA Normalization - Uses adaptive maximum normalization where

ela = ela /(ela.max() + 1e-8)

This prevents intensity saturation and preserves the relative forensic signals across different image types.

5. Training Strategy

5.1 Hyperparameters

Parameter	Value
Image Size	224 X 224
Batch Size	32
Epochs	30
Base Learning Rate	3e-4, 1e-5
Weight Decay	1e-4

Dropout	0.3, 0.15
Optimizer	AdamW
Scheduler	CosineAnnealingLR

5.2 Differential Learning Rate

Different parts of the network use different learning rates:

- **Backbone blocks (7,8)** – **1e-5** to carefully fine-tune pretrained semantic features.
- **SRM-CNN & ELA Branch** – **3e-4** since these layers are trained from scratch.
- **Fusion & Classifier** – **3e-4** for faster learning of cross-branch representations.

5.3 Loss Design

Weighted CrossEntropy – Chosen because tampering detection is a **binary classification task**, and cross-entropy is the standard loss for probabilistic classification. Applying class weights ensures the model remains **balanced between authentic and tampered samples**, preventing bias toward the majority class.
Label Smoothing (0.1) – Reduces over-confident predictions by slightly softening the target labels, which improves **model calibration and generalization**, particularly important for reliable forensic decision making.

5.4 Checkpointing

- **Last_checkpoint.pth** - Full training state for resuming.
- **best_model.pth** – Saved when validation accuracy improves.

5.5 Data Loading & Leakage Prevention

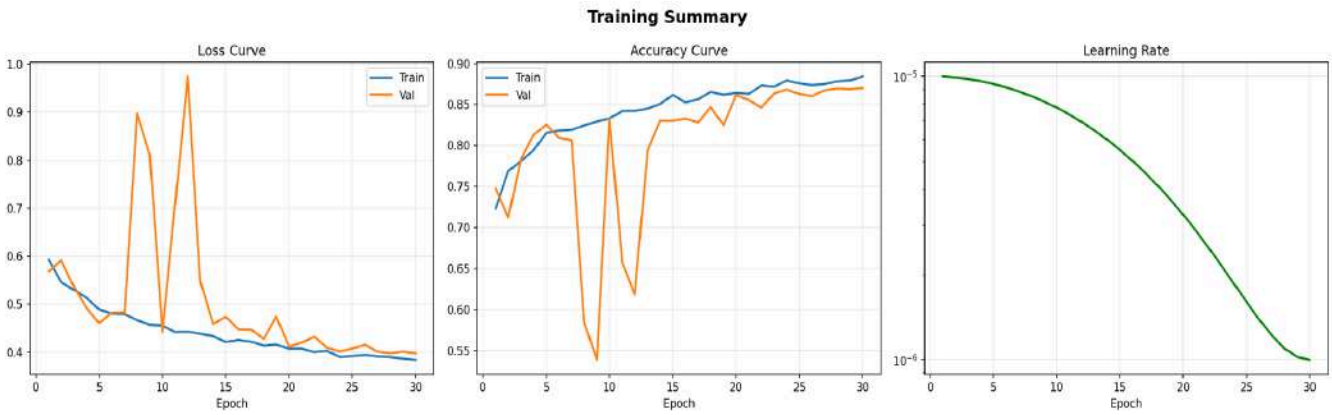
- Dataset stored on **local SSD in Google Colab** to reduce I/O latency and speed up training.
- Splits are performed using **unique** `original_image_path`, not tampered image paths.
- Each COCO image can generate multiple tampered versions; splitting by tampered images would cause **scene leakage**. Ensure no data leakage by using the above strategy.
- **70% / 15% / 15%** split (train / val / test) based on original image identity.
- Authentic source images are assigned to the **same split as their tampered variants**.
- **1:1 class balance** (tampered vs authentic) maintained in every split.

6. Evaluation & Results

6.1 Evaluation Metrics

Evaluation is conducted using a hollistic suite of metrics: **Accuracy, ROC-AUC, Precision, Recall,and F1-score**

6.2 Training Dynamics



After **30 epochs**, the model converged with stable training:

- **Train Accuracy:** 88.41%
- **Validation Accuracy:** 87.02%
- **Train Loss:** 0.3838
- **Validation Loss:** 0.3978

6.3 Test Performance

Evaluation on the held-out test set produced the following results:

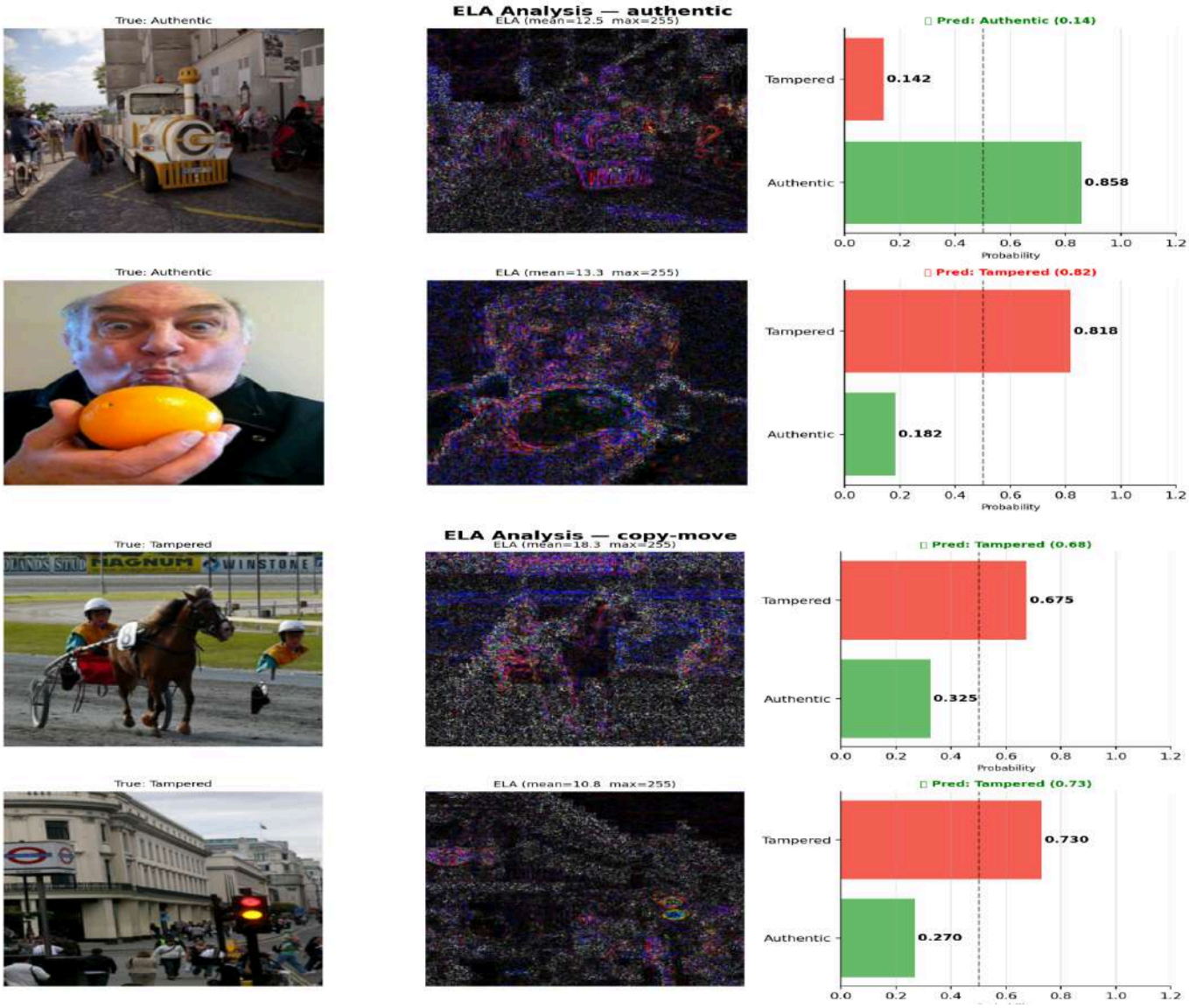
- **Overall Accuracy: 88.01%**
- **ROC-AUC Score: 0.9536**

```
Classification Report:
              precision    recall  f1-score   support

   Authentic       0.90      0.85      0.88       767
    Tampered       0.86      0.91      0.88       767

    accuracy                           0.88      1534
   macro avg       0.88      0.88      0.88      1534
weighted avg       0.88      0.88      0.88      1534
```

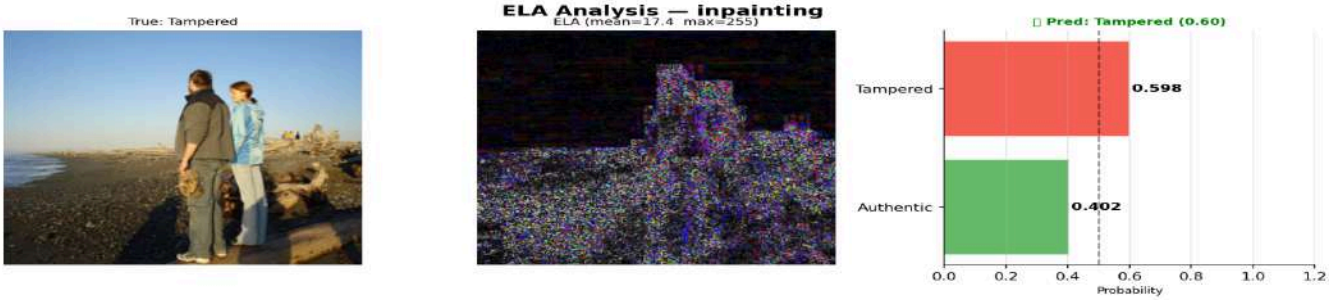
6.4 Per-Technique Accuracy

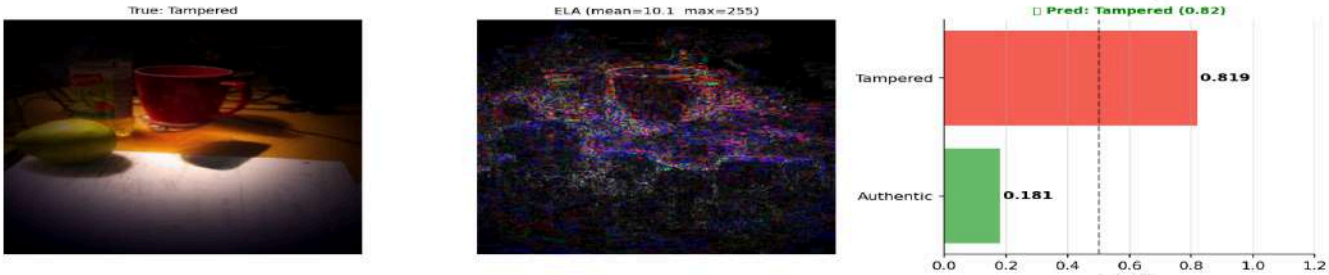
```
Per-Technique Accuracy:
Technique                    Acc  Correct  Total
--------------------------------------------------
authentic                 0.8581      508    592
authentic_bg              0.8400      147    175
copy-move                 0.9250      148    160
frequency_domain          0.9116      134    147
inpainting                0.9396      140    149
local_manipulation        0.8526      133    156
splicing                  0.9032      140    155
```

6.5 Results

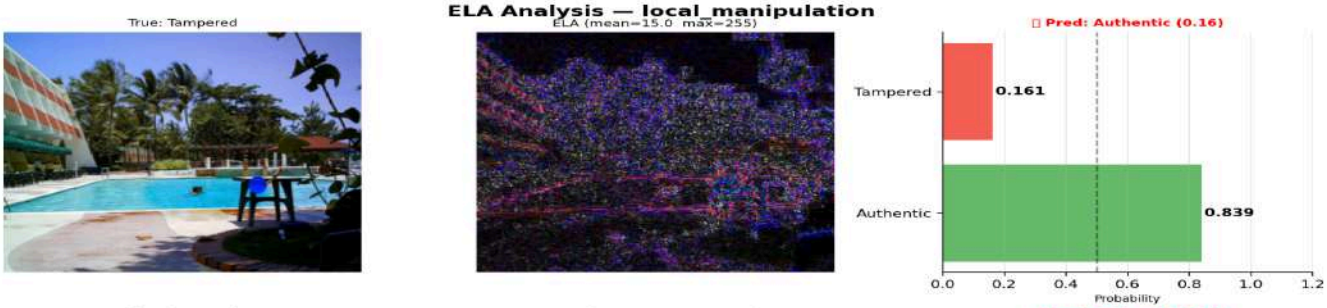

True: Authentic
ELA Analysis — authentic
ELA (mean=12.5 max=255)
Pred: Authentic (0.14)
Tampered
0.142
Authentic
0.858
0.0 0.2 0.4 0.6 0.8 1.0 1.2
Probability
True: Authentic
ELA (mean=13.3 max=255)
Pred: Tampered (0.82)
Tampered
0.818
Authentic
0.182
0.0 0.2 0.4 0.6 0.8 1.0 1.2
Probability
True: Tampered
ELA Analysis — copy-move
ELA (mean=18.3 max=255)
Pred: Tampered (0.68)
Tampered
0.675
Authentic
0.325
0.0 0.2 0.4 0.6 0.8 1.0 1.2
Probability
True: Tampered
ELA (mean=10.8 max=255)
Pred: Tampered (0.73)
Tampered
0.730
Authentic
0.270
0.0 0.2 0.4 0.6 0.8 1.0 1.2

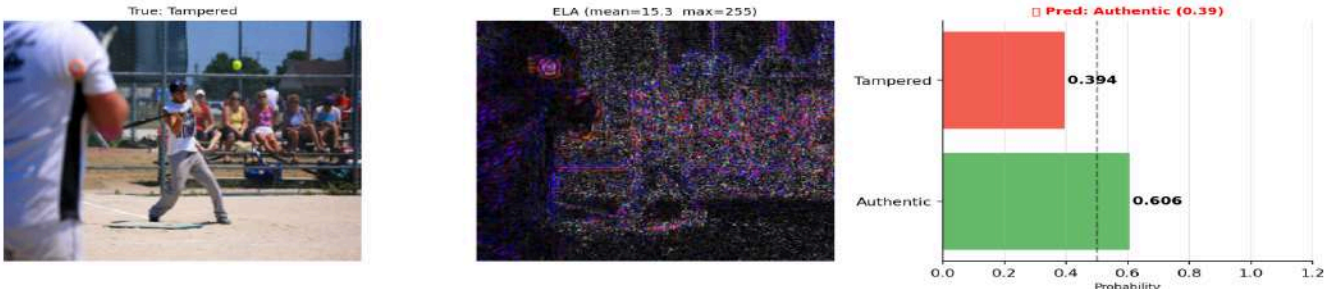
ELA Analysis — inpainting



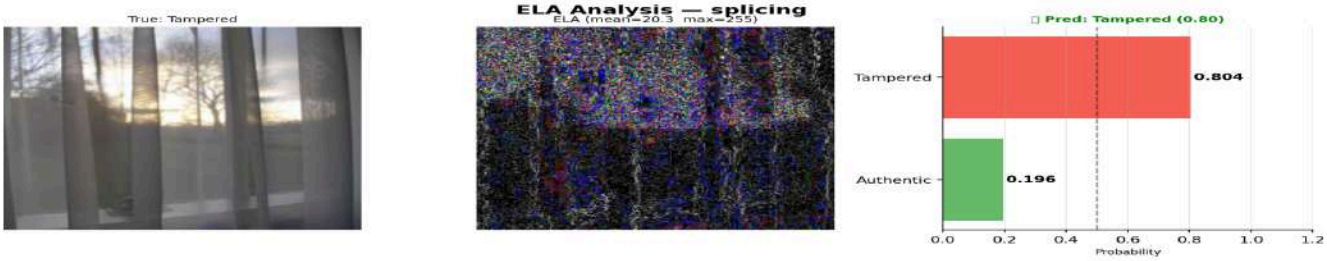


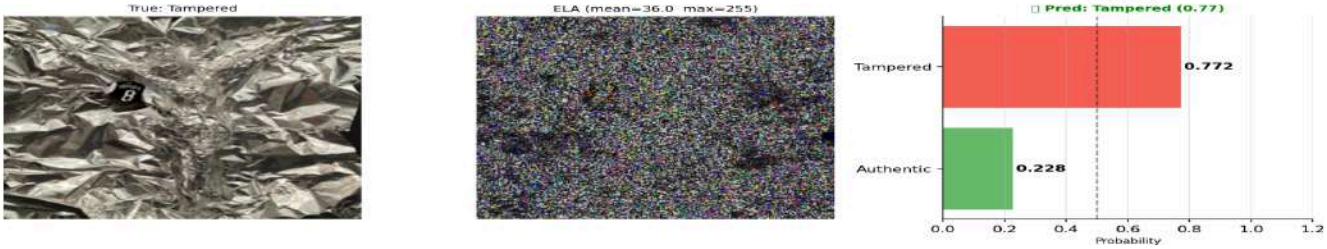
ELA Analysis — local_manipulation





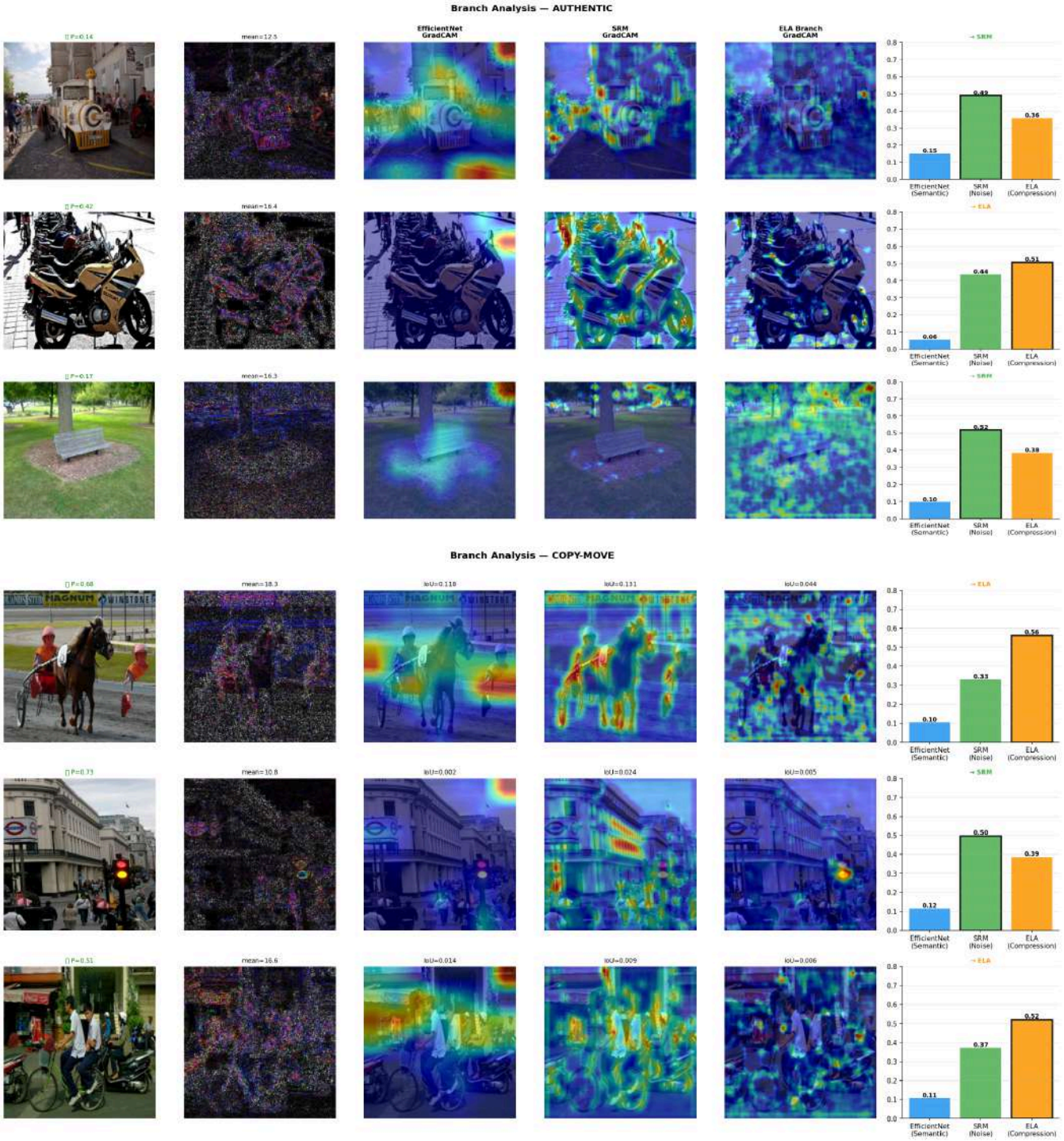
ELA Analysis — splicing



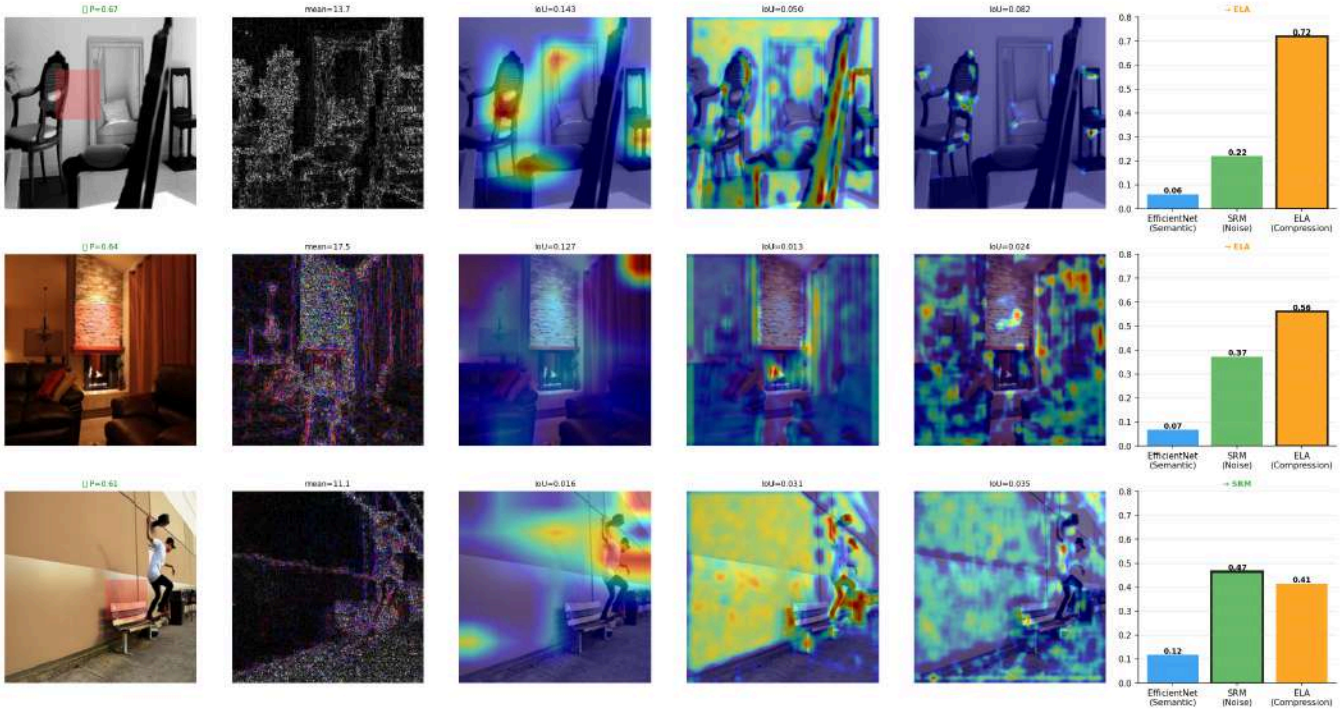


7. Explainability — GradCAM & Attention Analysis

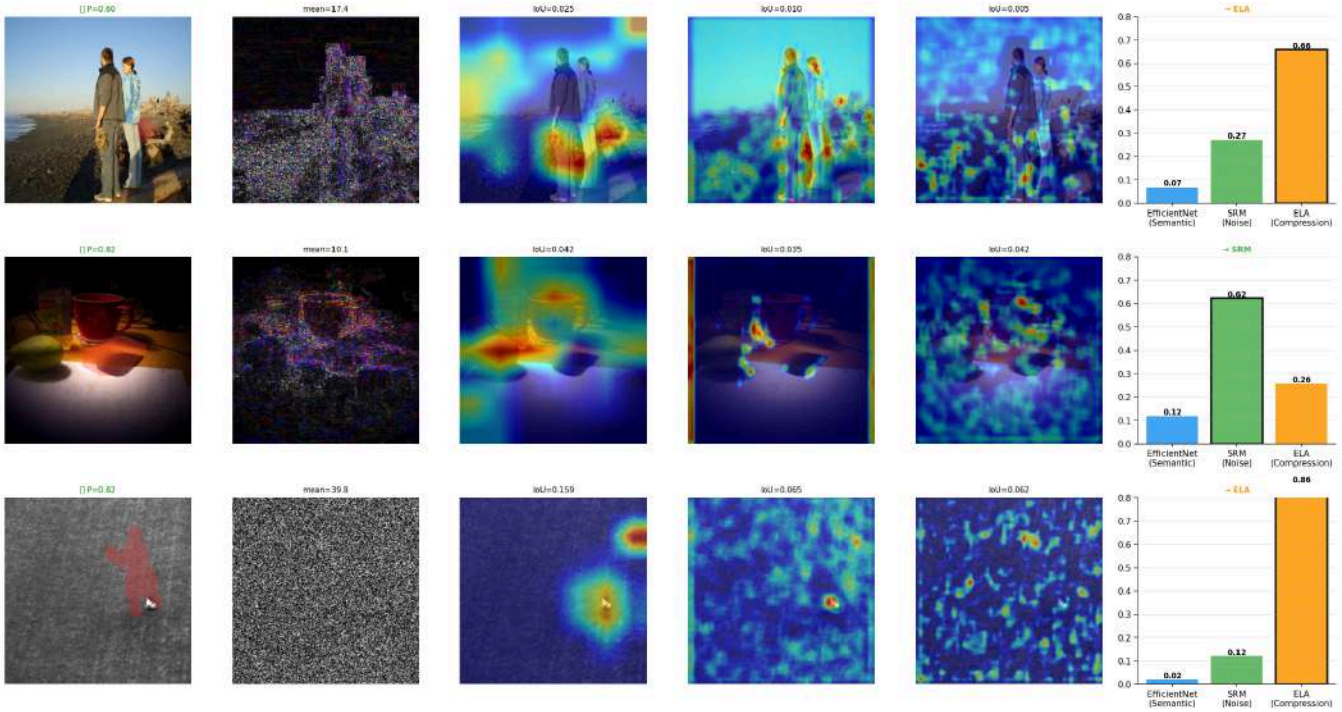
A custom visualization method used to generate heatmaps for the **Semantic, Noise, and ELA branches**. Hooks are registered on the final convolutional layers of each branch to highlight the regions that contribute most to the model's **"Tampered"** prediction. The heatmap is overlaid on the original image for interpretation, while the **red regions represent the ground-truth tampering mask from the dataset**, indicating the actual manipulated area.



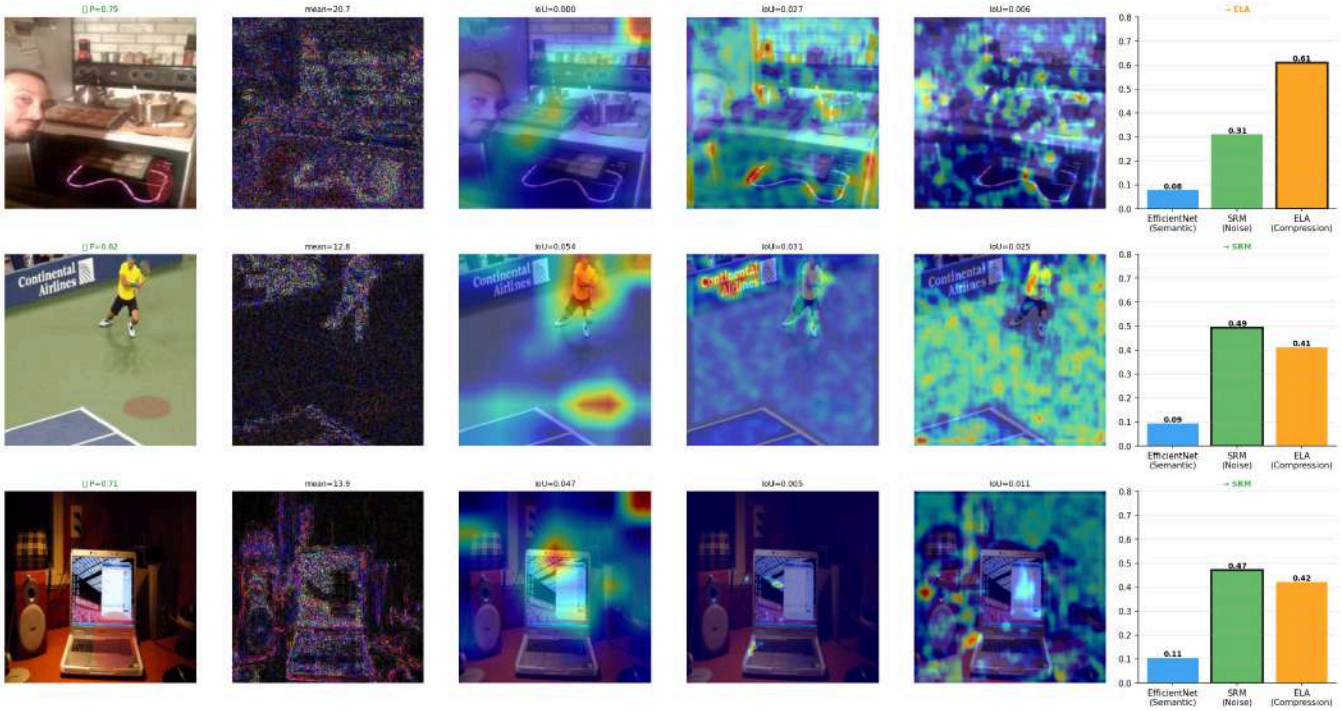
Branch Analysis — FREQUENCY_DOMAIN



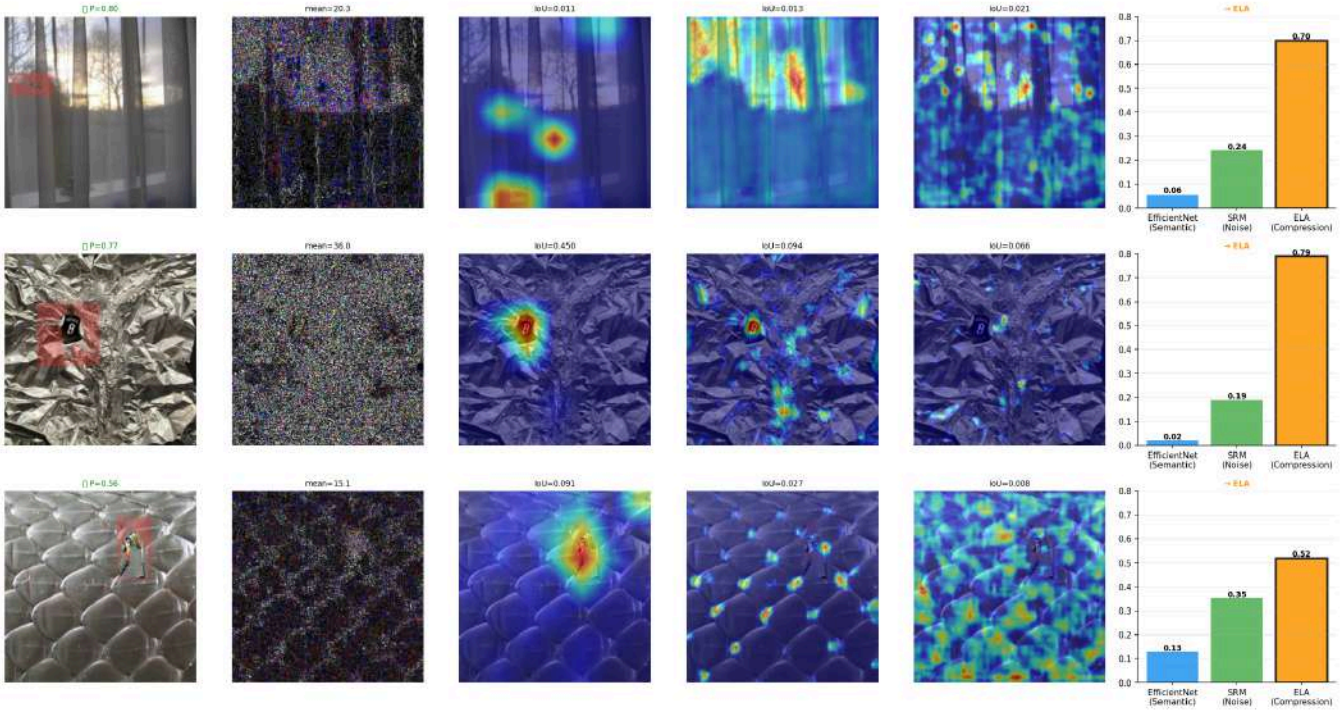
Branch Analysis — INPAINTING



Branch Analysis — LOCAL_MANIPULATION



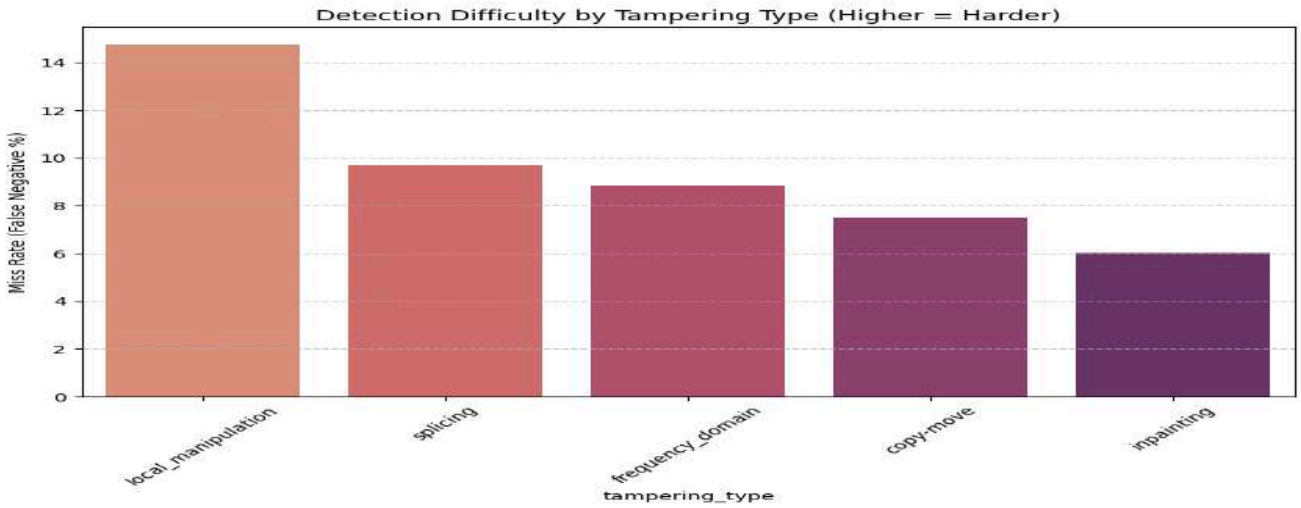
Branch Analysis — SPLICING



Using GradCAM-based contribution analysis, i examined which branch contributes most strongly for different tampering types.

Tampering Type	Dominant Branch	Forensic Rationale
Frequency Domain	ELA	DCT manipulations introduce strong JPEG compression artifacts
Splicing	ELA	Source images often have different compression histories
Copy-Move	ELA + SRM	Recompression and resampling alter both JPEG artifacts and noise patterns
Local Manipulation	SRM	Pixel-level edits disturb underlying sensor noise statistics
Inpainting	ELA + SRM	Interpolation disrupts both compression structure and noise consistency

8. Hardest Tampering Type Analysis: Local Manipulation



Local manipulation remains the most challenging technique to detect. Unlike splicing, it preserves the original noise statistics and rarely introduces strong JPEG mismatches. Furthermore, the manipulated region is often spatially small, allowing the authentic background to dominate the features during global pooling.

9. Cross-Dataset Generalization

When evaluated on the **IMD2020 dataset**, the model achieved **~40% accuracy**, which is significantly lower than the performance on the synthetic test set. This drop is primarily due to **domain shift**, as the training dataset contains controlled synthetic manipulations while IMD2020 consists of real-world edits with different camera pipelines, compression histories, and editing tools. This highlights the challenge of **cross-dataset generalization in image forensics**.

10. Limitations & Future Work

- **Domain Gap** – Real-world manipulations (e.g., IMD2020) often involve complex editing pipelines, camera processing, and multiple recompression stages, which differ from the controlled synthetic tampering used during training.
- **Generation Pipeline Bias** – The synthetic dataset is generated using a limited set of manipulation techniques and blending strategies. As a result, the model may partially learn **generation-specific artifacts** rather than universally transferable forensic cues.
- **Compression & Camera Pipeline Variability** – Real images pass through different **camera ISPs, demosaicing algorithms, and compression pipelines**, which can alter noise statistics and weaken the reliability of SRM-based residual features.
- **Localization Capability** – The current model performs **binary classification only**. Adding a **segmentation or localization head** could enable pixel-level tampering detection.
- **Small Region Sensitivity** – Global average pooling may dilute signals from **very small manipulated regions**. Future work could incorporate **region-level attention or patch-based analysis**.
- **Limited Manipulation Diversity** – Although multiple tampering types are included, real-world edits may involve **layered operations (resize → blur → recompress → clone)** which are not fully represented in the synthetic pipeline.
- **Cross-Dataset Generalization** – The model is trained primarily on **COCO-derived synthetic forgeries**, which limits its ability to generalize to datasets created with **different editing tools, resolutions, and compression histories**.